



INSTITUTO PROFESIONAL
SAN SEBASTIAN

Desarrollo de Aplicaciones Móviles

Ampliación, Depuración y Pruebas de Task Manager Mobile

Evaluación Unidad III

Docente: Boris Belmar Muñoz

Estudiantes:

Alonso García Espinoza

Enrique González Venegas

Fecha de Entrega: 06-09-2026

INFORME TÉCNICO APLICACIÓN MÓVIL DE GESTIÓN DE TAREAS

1. Introducción

El presente informe documenta la ampliación y validación de Task Manager Mobile, una aplicación para Android desarrollada con React Native, Expo y Typescript. La solución permite administrar tareas por usuario, trabajar sin conexión mediante almacenamiento local y sincronizar información con una API propia cuando existe conectividad. También incorpora cámara, geolocalización, fotografías, notas de voz locales e importación selectiva de tareas ficticias desde un servicio externo.

Aunque el enunciado propone Ionic y Capacitor como base tecnológica, el proyecto utiliza React Native con Expo, alternativa que cumple el mismo propósito funcional: construir una aplicación móvil con acceso a capacidades nativas, almacenamiento local, servicios web, autenticación y pruebas automatizadas. La decisión conserva el framework React elegido y utiliza módulos equivalentes del ecosistema Expo para cámara, ubicación, archivos, audio, almacenamiento seguro y compilación Android.

LINK A REPOSITORIO EN GITHUB:

<https://github.com/PROYECTOS-IPSS/to-do-list-ipss>

1.1. Objetivo

El objetivo fue transformar una aplicación básica de tareas en una solución móvil más estable y completa, capaz de:

- Capturar fotografías y asociarlas a una tarea.
- Registrar ubicación mediante GPS.
- Funcionar localmente sin conexión.
- Autenticar y aislar los datos con distintos usuarios.
- Sincronizar tareas e imágenes con un backend.
- Consumir una API externa.
- Aplicar validación, idempotencia y protección de datos.
- Ejecutar pruebas automatizadas y utilizar herramientas de depuración.

1.2. Arquitectura Implementada

Capa	Tecnología	Responsabilidad
Aplicación móvil	React Native, Expo SDK 57, Typescript y Expo Router	Interfaz, navegación y acceso a periféricos
Persistencia local	SQLite, SecureStore y filesystem	Tareas offline, sesión y archivos locales
API	Node.js, Express, Zod y JWT	Autenticación, validación, ownership y sincronización
Persistencia remota	PostgreSQL y Prisma	Usuarios, tareas, versiones y metadatos
Entorno	Docker compose y Expo Development Build	Servicios reproducibles y ejecución Android
Calidad	Jest, Testing Library y Supertest	Pruebas mobile, UI y backend

2. Configuración del Entorno de Prueba

Para ejecutar funciones nativas se utilizó Expo Development Build, ya que con Expo Go no representa adecuadamente todas las dependencias incorporadas. El entorno de desarrollo empleó Node.js 24.15.0, Yarn Classic 1.22.22, Expo SDK 57 y React Native 0.86.3.

El backend, las migraciones y PostgreSQL se iniciaron mediante Docker Compose. Metro se ejecutó en el computador y el dispositivo se conectó a través de la misma red local. La URL pública se configuró con la dirección LAN del host, evitando 'localhost', que desde Android apunta al propio dispositivo.

2.1. Entornos utilizados

Para el desarrollo de la app, se generó un Development Build y se instaló en un Samsung S25+ con la versión de Android 16, conectándose mediante la red local con el computador corriendo el backend y Metro. De esta forma se podía revisar de manera más cómoda el funcionamiento correcto de funciones como cámara, GPS, audio, almacenamiento local, sincronización, etc.

```
</> Bash

mkdir -p eas-local-work/tmp eas-local-work/work build-output

cd mobile

TMPDIR="$PWD/../../eas-local-work/tmp" \
EAS_LOCAL_BUILD_WORKINGDIR="$PWD/../../eas-local-work/work" \
EAS_LOCAL_BUILD_ARTIFACTS_DIR="$PWD/../../build-output" \
eas build --platform android --profile development --local
```

Utilizando estos comandos, se puede generar un .apk dentro de la raíz del proyecto, si no se ha iniciado sesión con Expo se puede usar *eas login* para iniciar sesión y producir la build de manera local.

Al terminar, el .apk debería quedar en *build-output/* y luego puedes pasar el apk a tu celular, instalarlo y seguir los pasos de README.md.

3. Funcionalidades Desarrolladas

3.1. Periféricos y almacenamiento local

La cámara permite capturar una fotografía, mostrar una vista previa, copiar el archivo desde caché hacia almacenamiento persistente y asociar su metadata a la tarea. Las imágenes se muestran inmediatamente y pueden sincronizarse posteriormente. El GPS registra latitud, longitud y precisión, validando que el conjunto de ubicación este completo antes de persistirlo.

Las tareas se guardan primero en SQLite con una identidad local estable. Esto permite crear, editar, completar y eliminar registros sin conexión. Al recuperar conectividad, una cola durable sincroniza los cambios con PostgreSQL.

Las notas de voz se graban y reproducen localmente; su sincronización remota quedó fuera del alcance estable para evitar afirmar una función no implementada correctamente.

Crear una tarea

Anota lo siguiente que quieres sacar adelante.

Título

Test

Descripción

Descripción de testeo

Asociar ubicación actual

Ubicación asociada

-53.1274262, -70.8699545

Precisión 13 m · 5/9/2026, 5:21:00 p.m.

Quitar ubicación

Añadir fotografía



Asociar ubicación actual

Añadir fotografía

Añadir tarea

Tus tareas

Sincronizar

Pendientes

Todas

Completadas

Test

Pendiente

Descripción de testeo

5/9/2026 · Ubicación · precisión 13 m



Ver detalles

Completar

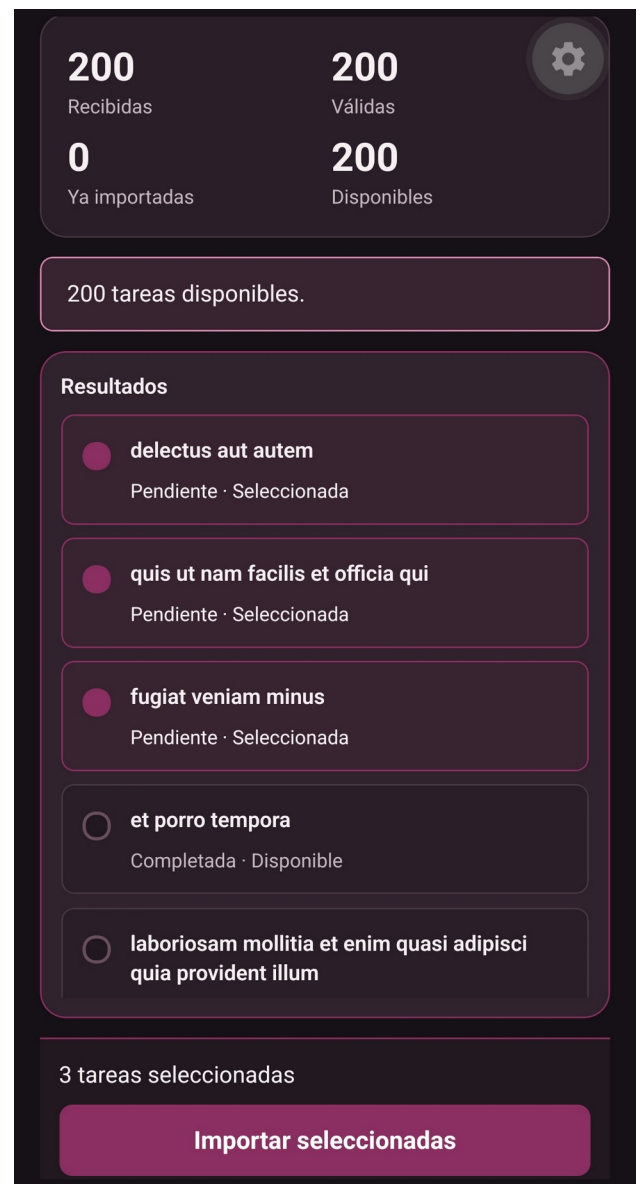
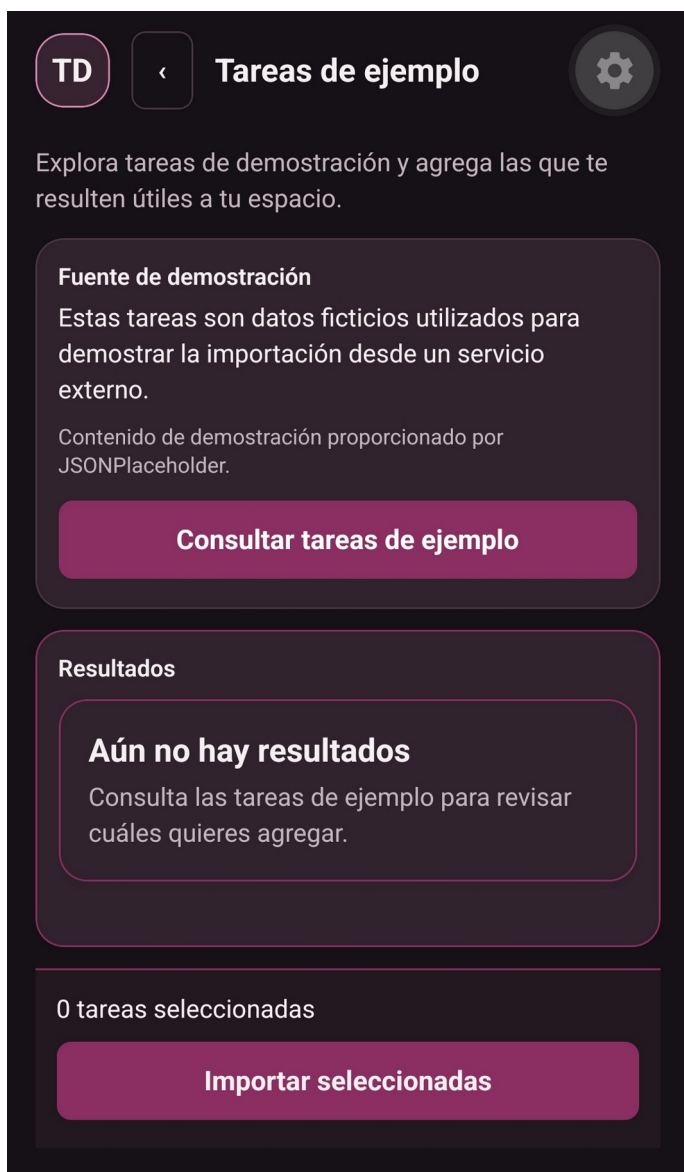
Editar

Eliminar

3.2. Servicios web y API externa

La API propia implementa registro, login, tareas, imágenes, audio backend, health y readiness. Las operaciones protegidas utilizan JWT en el encabezado 'Authorization'. La sincronización de tareas emplea identificadores idempotentes y control de versión con 'If-Match', evitando duplicados y haciendo visibles los conflictos.

La pantalla "Tareas de ejemplo" consume registros ficticios de JSONPlaceholder, los valida y normaliza antes de mostrarlos. El usuario selecciona cuáles importar y el sistema conserva su procedencia local para impedir importaciones duplicadas.

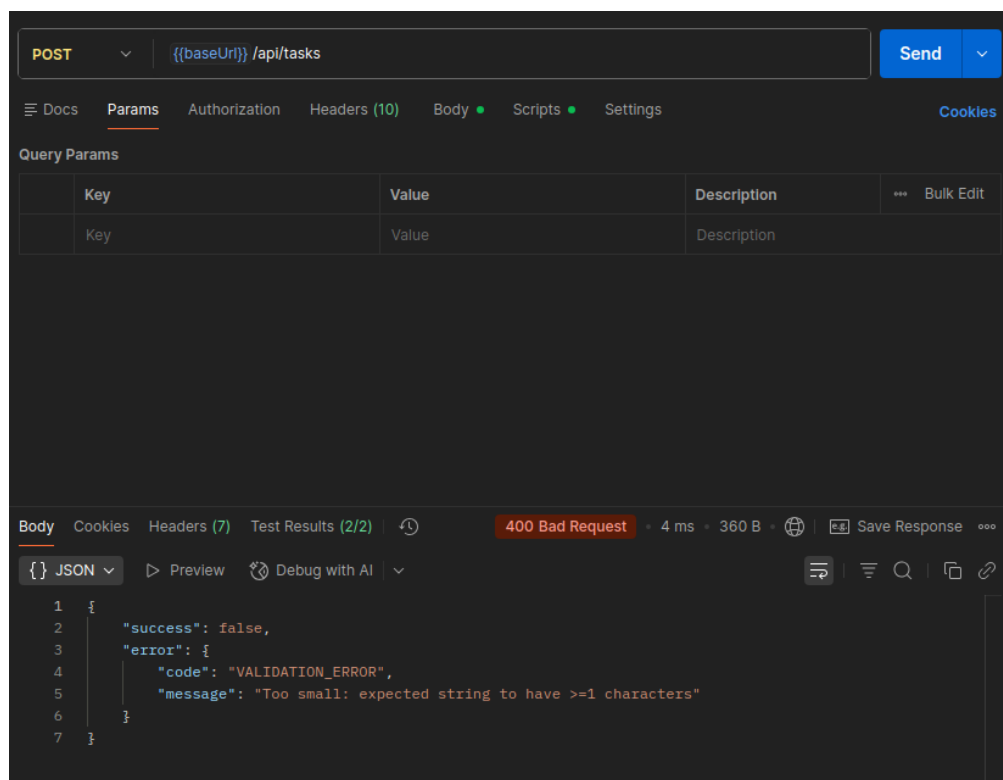


3.3. Seguridad y protección de datos

Las principales medidas aplicadas fueron:

- Contraseñas almacenadas como hashes con bcryptjs.
- Autenticación mediante JWT.
- Token conservado en SecureStore y enviado solo por encabezado Bearer.
- Validación Zod en los límites HTTP.
- Aislamiento de tareas y adjuntos según el usuario autenticado.
- Rutas protegidas para descargar archivos, sin exposición pública de '/uploads'.
- Secretos en un '.env' ignorados por Git.
- Consultas parametrizadas a través de Prisma.
- Respuestas 404 para recursos ajenos, evitando revelar su existencia.
- Límites de tamaño y MIME de archivos.

Ejemplo:



4. Depuración y Resolución de Problemas

Durante el desarrollo se combinaron logs de Metro, consola de Android, respuestas HTTP, Docker Compose, PostgreSQL y pruebas focalizadas. Este proceso permitió aislar errores en la frontera entre interfaz, filesystem, red y backend.

Problema detectado	Evidencia utilizada	Causa identificada	Solución aplicada
Mutaciones devolvían 401	Logs de auth y API	Headers adicionales reemplazaban "Authorization"	Combinación segura de encabezados Bearer e idempotencia
Tareas reaparecían o mostraban keys duplicadas	Warning React y estado SQLite	Operaciones huérfanas e inserción repetida	Identidad "localId", cancelación transaccional y deduplicación
Fotografía no aparecía inmediatamente	Navegación Home/detalle	Carrera entre snapshot de imágenes y persistencia	Invalidación de lectura anterior y publicación de URI definitiva
Upload Android fallaba	Mensaje "Unsupported FormDataPart implementation"	Parte multipart incompatible con transporte nativo	Archivo de Expo y transporte multipart compatible
Imagen no sincronizada no cargaba	PostgreSQL, filesystem y HTTP 404	Metadata existía, pero descarga pública no era válida	Endpoint protegido, URL de contenido y reconciliación local/remota
Lista importable no permitía navegar correctamente	Validación física y warnings de Metro	Conflictos entre scrolls virtualizados	Componente acotado con scroll interno no virtualizado

La depuración no se limitó a ocultar mensajes. Cada corrección preservó el error observable, añadió una regresión automatizada cuando correspondía y confirmó que no se modificaran contratos ajenos.

4.1. Herramientas empleadas

- Metro y consola de React Native
- Android Studio y Logcat.
- ADB: detección, instalación y ejecución del cliente.
- Postman y 'curl': contratos HTTP y archivos protegidos.
- Docker Compose: estado y logs de backend/base de datos.
- Prisma Studio y 'psql': contraste de metadata y persistencia.
- Jest, Testing Library y Supertest: reproducción controlada y regresiones.

5. Pruebas Automatizadas y Resultados

La calidad continua se verificó mediante pruebas unitarias, de integración y de componentes. El backend se probó con Jest y Supertest; la aplicación móvil utilizó Jest, Testing Library y SQLite real en las suites de persistencia. Las pruebas abarcaron autenticación, ownership, tareas, ubicación, imágenes, idempotencia, sincronización, importación, preferencias y componentes accesibles.

5.1. Comandos de verificación

```
yarn typecheck
yarn typecheck:tests
yarn lint
yarn test
```

5.2. Resultados finales

Verificación	Resultado
Backend	8 suites / 80 tests aprobados
Aplicación móvil	14 suites / 127 tests aprobados
Total	22 suites / 207 tests aprobados
Typecheck producto	Aprobado
Typecheck de tests	Aprobado
ESLint	Aprobado



```
● bash-5.3$ yarn typecheck
yarn typecheck:tests
yarn lint
yarn test
yarn run v1.22.22
$ yarn workspace task-manager-mobile typecheck && yarn workspa
$ tsc --noEmit
$ tsc --noEmit
Done in 3.08s.
yarn run v1.22.22
$ yarn workspace task-manager-backend typecheck:tests
$ tsc -p tsconfig.test.json --noEmit
Done in 1.56s.
yarn run v1.22.22
$ eslint --ignore-pattern .yarn --ignore-pattern mobile/dist .
Done in 1.43s.
yarn run v1.22.22
$ yarn test:backend && yarn test:mobile
$ yarn workspace task-manager-backend test
$ jest --runInBand
PASS tests/attachment.test.ts
PASS tests/auth.test.ts
PASS tests/task.test.ts
PASS tests/peripherals.test.ts
PASS tests/image-file.test.ts
PASS tests/preferences.test.ts
PASS tests/server.test.ts
PASS tests/location.test.ts

Test Suites: 8 passed, 8 total
Tests:      80 passed, 80 total
Snapshots:  0 total
Time:       1.683 s, estimated 5 s
Ran all test suites.
$ yarn workspace task-manager-mobile test
$ jest --runInBand
PASS src/services/__tests__/task-repository.test.ts
PASS src/ui/__tests__/authenticated-image.test.tsx
PASS src/ui/__tests__/import-components.test.tsx
PASS src/services/__tests__/sync-service.test.ts
PASS src/services/__tests__/task-composer.test.tsx
PASS src/services/__tests__/auth.schemas.test.ts
PASS src/auth/__tests__/AuthProvider.test.tsx
PASS src/services/__tests__/jsonplaceholder-adapter.test.ts
PASS src/services/__tests__/auth.test.ts
```

```

● Console

console.info
[auth] start GET /api/auth/me http://10.205.187.96:3000

at info (src/services/auth.ts:40:54)

console.info
[auth] response GET /api/auth/me 200 5ms

at info (src/services/auth.ts:47:56)

console.info
[auth] start GET /api/auth/me http://10.205.187.96:3000

at info (src/services/auth.ts:40:54)

console.info
[auth] response GET /api/auth/me 401 0ms

at info (src/services/auth.ts:47:56)

console.info
[auth] start GET /api/auth/me http://10.205.187.96:3000

at info (src/services/auth.ts:40:54)

console.info
[auth] response GET /api/auth/me 200 0ms

at info (src/services/auth.ts:47:56)

console.info
[auth] start GET /api/auth/me http://10.205.187.96:3000

at info (src/services/auth.ts:40:54)

console.info
[auth] start GET /api/auth/me http://10.205.187.96:3000

at info (src/services/auth.ts:40:54)

PASS src/services/__tests__/image-sources.test.ts
PASS src/services/__tests__/tasks.test.ts
PASS src/services/__tests__/attachments.test.ts
PASS src/services/__tests__/photo-persistence.test.ts
PASS src/services/__tests__/preferences.test.ts

Test Suites: 14 passed, 14 total
Tests: 127 passed, 127 total
Snapshots: 0 total
Time: 2.159 s, estimated 6 s
Ran all test suites.
Done in 5.24s.

```

Las pruebas automatizadas verifican la lógica y contratos, pero no sustituyen la validación del bridge nativo. Cámara, GPS, audio, permisos, rotación y gestos también fueron comprobados.

5.3. Cobertura del criterio de evaluación

Indicador de la rúbrica	Evidencia presentada
Configuración de emuladores y escenarios	Tablas de entornos, AVD Manager y pruebas de compatibilidad
Técnicas de depuración	Casos diagnosticados mediante Metro, Logcat, API, Docker y base de datos
Pruebas automatizadas	22 suites y 207 tests; typecheck y lint integrados en verificación final

6. Conclusiones

La ampliación de Task Manager Mobile permitió integrar periféricos, persistencia offline, servicios web, autenticación y pruebas automatizadas en una solución coherente. La separación entre SQLite y PostgreSQL mantiene la aplicación utilizable sin conexión, mientras la sincronización idempotente reduce duplicados y conserva conflictos para revisión explícita.

La depuración basada en evidencia fue fundamental para resolver problemas que involucraban simultáneamente estado local, transporte HTTP, filesystem y renderizado. El uso de pruebas de regresión permitió verificar que las correcciones no reintrodujeran fallos anteriores y que los contratos de seguridad permanecieran intactos.

Finalmente el proyecto cumple funcionalmente los objetivos centrales de la unidad mediante tecnologías equivalentes del ecosistema React Native/Expo.